

Large Language Models (LLMs) Part 1: Tokenization and Embedding

Ronald A. Sumida

rsumida@innovationinsoftware.com



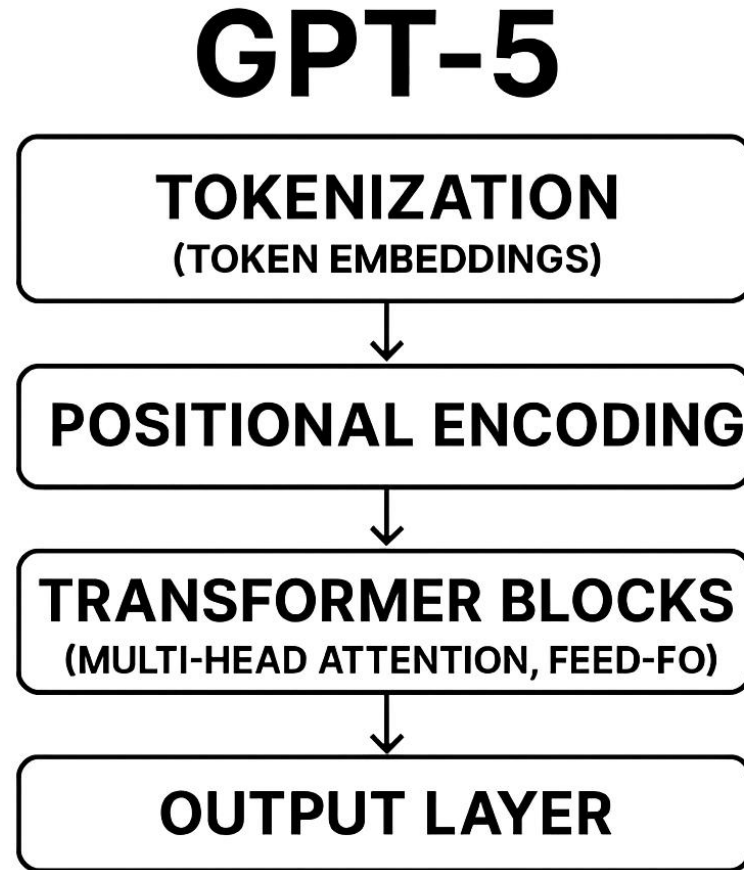
Outline

- Tokens
 - Text
 - Sequences
 - Removing
 - Stop words
 - Numbers
- Embeddings
 - Definition
 - Generation
 - Usage

What is LLM/GPT?

- Large Language Model (LLM)
 - Capable of performing variety of NLP tasks
 - Trained with “large” number of parameters
 - Learns from trillions of tokens representing enormous amounts of linguistic data
- Generative Pre-trained Transformer (GPT)
 - Type of LLM using transformer architecture
 - Encoder
 - Generates vector sequence called *embeddings* from input text
 - Decoder
 - Predicts next word from previous words and embeddings
 - Uses *attention* to focus on most significant parts of input and output

GPT-5 Architectural Diagram



Tokens

- Generated by splitting unstructured text into smaller units
 - Processed by LLMs for NLP
- Can represent
 - Characters
 - Words/subwords
 - Phrases
 - Segments of text/code
- Exact representation depends upon tokenization strategy
- Simple tokenization example
 - <https://platform.openai.com/tokenizer>

Preparing Text for NLP

- **Step 1:** Prepare text for tokenization
 - Remove stop words and numbers (optional)
 - Stem or lemmatize words — e.g., turn "cats" into "cat" (optional)
- **Step 2:** Tokenize text
 - Converts text to lowercase and removes symbols
 - Splits sentences into words and represents each word with an integer index
 - E.g., use Keras's **Tokenizer** class or equivalent
- **Step 3:** Create padded sequences from tokenized word indexes
 - Ensures that each sample input to the network is of the same length
 - Keras's **pad_sequences** method does this for you

Tokenizing Text example

```
from tensorflow.keras.preprocessing.text import Tokenizer
from keras.utils import pad_sequences
lines = ['Quick brown fox', # Stop words removed
        'Jumps over lazy lazy brown dog']
tokenizer = Tokenizer()
tokenizer.fit_on_texts(lines)
vectors = tokenizer.texts_to_matrix(lines)
tokenizer.word_index
```

vectors



	brow n	lazy	quick	fox	jumps	over	dog
0	1	0	1	1	0	0	0
0	1	1	0	0	1	1	1

Vocabulary

brown	1
lazy	2
quick	3
fox	4
jumps	5
over	6
dog	7

Turning Text into Sequences

```
lines = ['Quick brown fox', # Stop words removed  
        'Jumps over lazy lazy brown dog']
```

```
tokenizer = Tokenizer()  
tokenizer.fit_on_texts(lines)  
sequences = tokenizer.texts_to_sequences(lines)  
padded_sequences = pad_sequences(sequences, 5)  
tokenizer.word_index
```

padded_sequences



0	0	3	1	4
6	2	2	1	7

Padded Sequences

Vocabulary

brown	1
lazy	2
quick	3
fox	4
jumps	5
over	6
dog	7

5

Removing Stop Words

- **Stop words** common words with no/minimal meaning (e.g., "the", "a")
- **Tokenizer** converts text to lowercase and remove symbols,
 - Doesn't remove stop words or words containing numbers
- Use **NLTK** or other libraries if you want stop words removed

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
```

```
def remove_stop_words(text):
    text = word_tokenize(text.lower())
    stop_words = set(stopwords.words('english'))
    # Remove stop words AND words containing numbers
    text = [word for word in text if word.isalpha() and not word in stop_words]
    return ' '.join(text)
lines = list(map(remove_stop_words, lines))
```

Removing Numbers

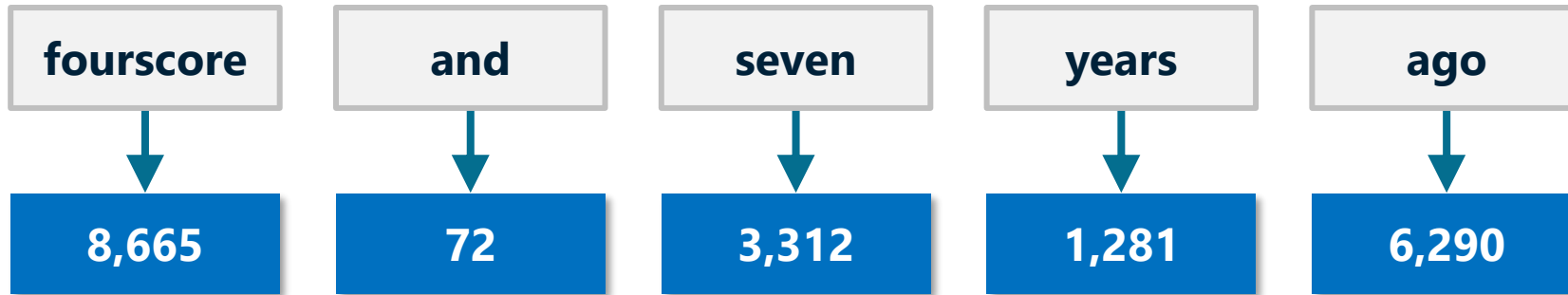
- **Filters** parameter removes numbers from tokenized text
 - Doesn't remove the words containing them

```
tokenizer = Tokenizer(filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n0123456789')
```

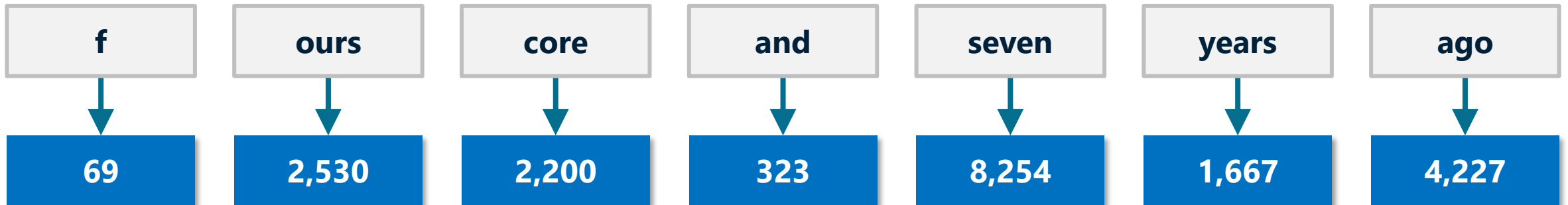
```
tokenizer.fit_on_texts(lines)  
sequences = tokenizer.texts_to_sequences(lines)  
padded_sequences = pad_sequences(sequences, 5)
```

Encoding

Word-for-Word Encoding



Byte-Pair Encoding (Subword Tokenization)



Byte-Pair Encoding (BPE)

- Tokenizer used for GPT models by OpenAI and Azure OpenAI
- Data compression algorithm
 - First published in 1994 article in *Dr. Dobbs's Journal*
- Begin compression
 - Replace most common byte pair with a special byte not occurring in data
 - Repeat
- End compression
 - When there are no byte pairs that occur more than once
 - Text is effectively compressed at this point
- Decompression
 - Reverse process by replacing special byte with byte pair

Byte-Pair Encoding (BPE) example

- Input text: *aaabdaaabc*
- Step 1: Replace most frequent pair *aa* with *Z*
 - *ZabdZabc*
 - *Z=aa*
- Step 2: Replace next most frequent pair *ab* with *Y*
 - *ZYdZYac*
 - *Y=ab*
 - *Z=aa*
- Step 3: No original byte pairs occurring more than once
 - Can stop with final result *ZYdZYac*
 - Can continue recursively by replacing *ZY* with *X* for final result *XdXac*
- Example from https://en.wikipedia.org/wiki/Byte_pair_encoding

Tiktoken

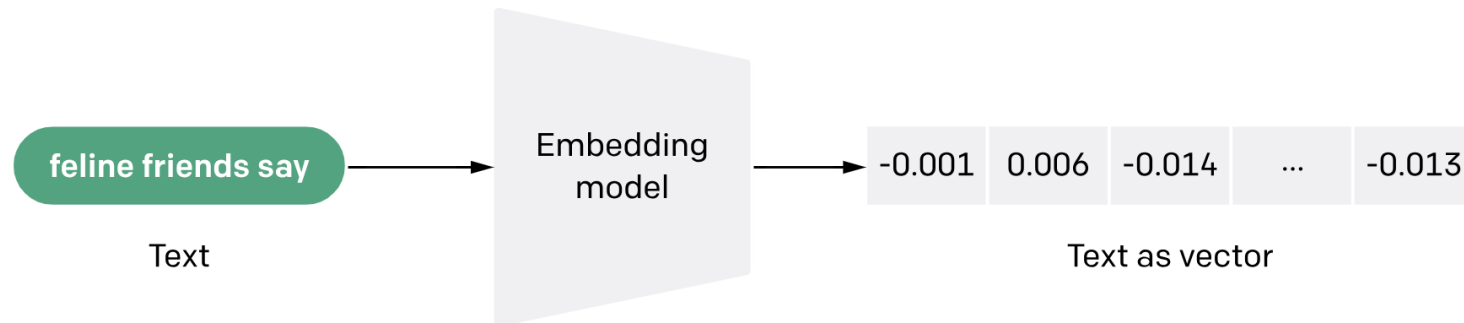
- Variant of BPE tokenizer
 - Used by OpenAI models
 - *cl100k_base*
 - Used since gpt-3.5-turbo
 - *R50k_base*
 - Used by older GPT-3 models
- Fast
 - 3-6x faster than open-source tokenizers
- <https://github.com/openai/tiktoken>

Tiktoken example

- Merges byte-level tokens based on frequency and model optimization
- Input text: *aaabdaaabc*
- Step 1: *aaab* recognized as frequent token
- Step 2: *d* is a separate token
- Step 3: *aaab* again recognized as frequent token
- Step 4: *ac* possibly merged if frequent

Embedding

- Encodes tokens as vector of floating-point numbers
 - Dimensions can correspond to features or attributes
- <https://openai.com/blog/introducing-text-and-code-embeddings>



Classifying Text

- How do you build machine-learning models that classify text?
 - Classify e-mails as spam or not spam
 - Score text for sentiment (sentiment analysis)
 - Analyze documents and categorize them by type or topic
- Raw text must be prepared for use in machine learning
 - Remove numbers, symbols, punctuation characters, and optionally stop words (e.g., "and" and "the") and convert to lowercase
 - Optionally stem or lemmatize words (e.g., turn "cats" into "cat")
- Traditional ML models typically require *vectorization* before training

Vectorizing Text

- Scikit provides three powerful text vectorizers
 - **CountVectorizer** creates a dictionary ("vocabulary") from a corpus of words provided to it and converts individual documents (sequences of words) into sparse vectors whose indexes correspond to vocabulary terms and whose values represent word counts.
 - **HashingVectorizer** does the same but conserves memory by storing hashes rather than words in the dictionary
 - **TfidfVectorizer** creates a dictionary from words provided to it and converts individual documents into vectors containing Term Frequency-Inverse Document Frequency (TFIDF) values between 0.0 and 1.0 reflecting the relative importance of individual words
- All three vectorizers include the ability to prepare text as well

Using CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer
documents = ['The quick brown fox',
             'Jumps over the lazy lazy brown dog']
vectorizer = CountVectorizer(stop_words=['the'])
word_matrix = vectorizer.fit_transform(documents)
```

vectorizer.vocabulary_

word_matrix.toarray()



brown	dog	fox	jumps	lazy	over	quick
1	0	1	0	0	0	1
1	1	0	1	2	1	0

Vocabulary

quick	6
brown	0
fox	2
jumps	3
over	5
lazy	4
dog	1

Demo

Sentiment Analysis



Keyword Extraction

- Extract keywords from documents to summarize their content
 - Term frequency (TF)
 - Count how many times a word appears in a single document
 - Inverse Document Frequency (IDF)
 - Measure how common or rare a word is across all documents
- Vectorize with **TfidfVectorizer** and then:
 - Pick the words or n -grams with the highest maximum TFIDF scores, or
 - Sum the TFIDF scores for individual words or n -grams and select the ones that yield the highest sums
- Values chosen for **min_df** and **max_df** parameters to **TfidfVectorizer** can greatly affect the outcome
 - Experiment with different values to optimize the model

CountVectorizer vs. TfidfVectorizer

```
documents = ['The quick brown fox',  
            'Jumps over the lazy lazy brown dog']
```

```
vectorizer = CountVectorizer(stop_words=['the'])  
word_matrix = vectorizer.fit_transform(documents)
```

	brown	dog	fox	jumps	lazy	over	quick
0	1	0	1	0	0	0	1
1	1	1	0	1	2	1	0

```
vectorizer = TfidfVectorizer(stop_words=['the'])  
word_matrix = vectorizer.fit_transform(documents)
```

	brown	dog	fox	jumps	lazy	over	quick
0	0.449436	0.000000	0.631667	0.000000	0.000000	0.000000	0.631667
1	0.259698	0.364996	0.000000	0.364996	0.729993	0.364996	0.000000

Sorted by Relevance

(4, 0.7299929362895164) => lazy
(2, 0.6316672017376245) => fox
(6, 0.6316672017376245) => quick
(0, 0.4494364165239821) => brown
(1, 0.3649964681447582) => dog
(5, 0.3649964681447582) => over
(3, 0.3649964681447582) => jumps
(0, 0.25969799324016246) => brown



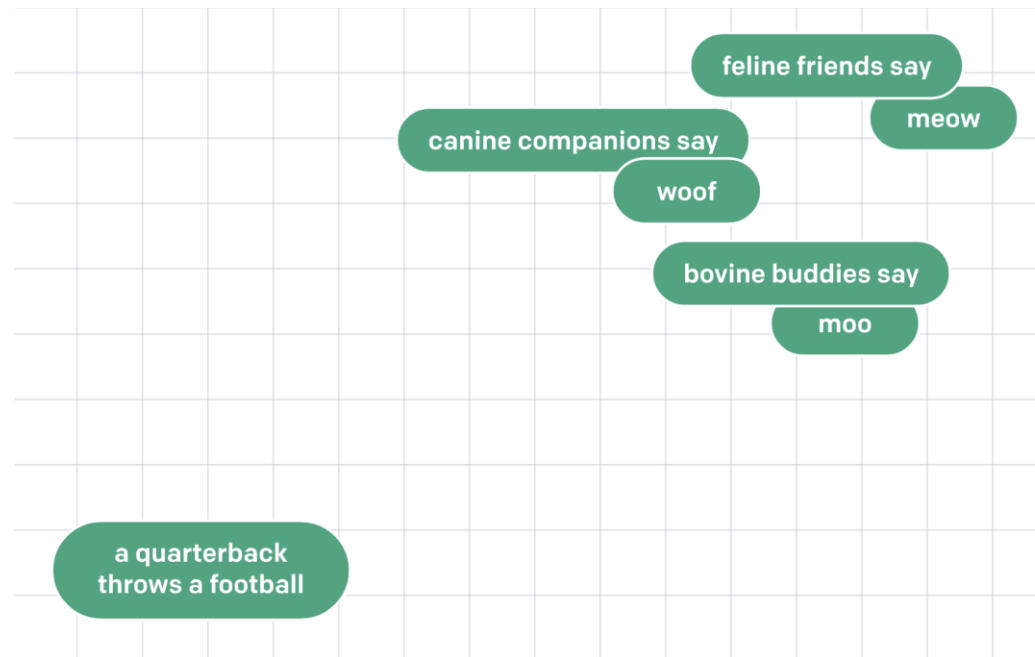
Demo

Keyword Extraction



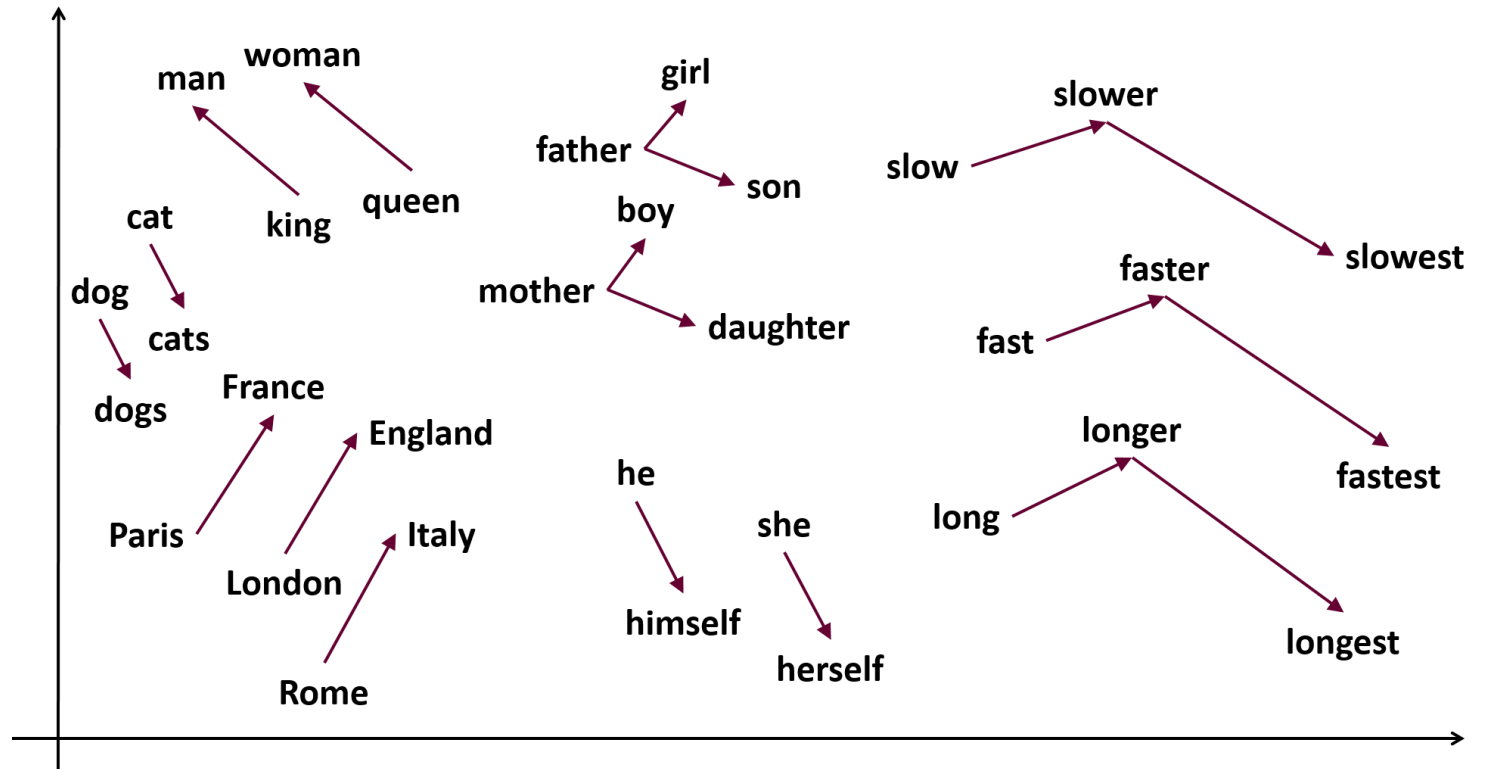
Semantic Embeddings

- Representation corresponds to semantic meaning of text
 - Similar texts represented by similar vectors
 - Distance between vectors in space reflects similarity



Word Embeddings

- Treat each word embedding as a vector in m -dimensional space
- Related words share proximity in embedding space
- Directions of travel in embedding space represent semantic paths between words



Source: <https://peach.ebu.io/docs/technical/tutorials/tuto-embeddings/>

Embeddings API

- Generates high-quality embedding vectors from text
- Earlier versions
 - Sensitive to text length
 - Used separate models for short vs. long text samples
- *text-embedding-ada-002*
 - Legacy model that is still supported
- *text-embedding-3-small*
 - Efficient and fast
- *text-embedding-3-large*
 - Highest accuracy and semantic richness

Embeddings API Model comparison

- *text-embedding-ada-002*
 - Generates vectors of 1,536 floating-point numbers
 - Recommended to update to version 3 models
- *text-embedding-3-small*
 - Generates vectors of 1,536 floating-point numbers
 - Applicable to
 - Real-time applications
 - Cost sensitive scenarios
- *text-embedding-3-large*
 - Generates vectors of up to 3,072 floating-point numbers
 - Applicable to
 - Semantic search
 - RAG
 - Clustering

Generating an Embedding Vector

```
from openai import OpenAI

client = OpenAI()

# openai.Embedding.create first performs tokenization using BPE
# Generates embedding vectors from tokens
response = client.embeddings.create(
    model='text-embedding-3-large',
    input='Four score and seven years ago our fathers brought forth, ' \
        'upon this continent, a new nation, conceived in liberty, and ' \
        'dedicated to the proposition that all men are created equal'
)

embedding = response.data[0].embedding
embedding
```

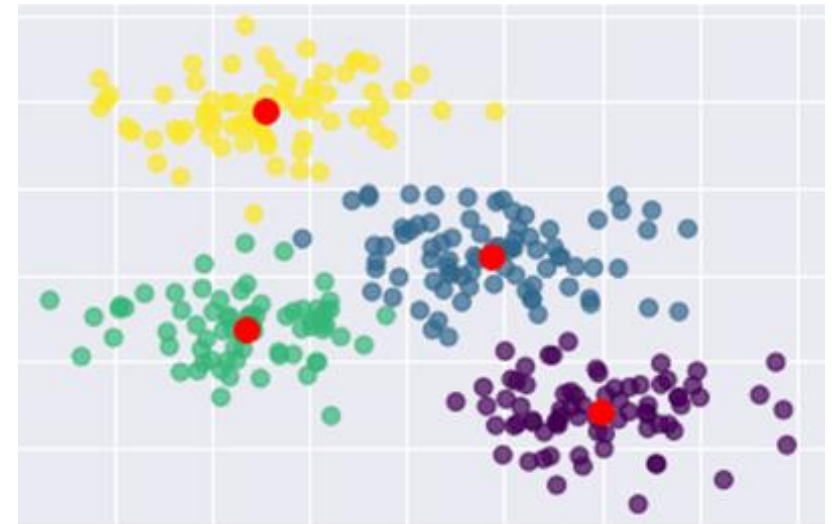
Supervised vs. Unsupervised Learning

- Supervised learning
 - Learn from labeled data in order to perform predictive analytics
 - For example, predict whether an e-mail is spam or predict the cost of a taxi ride or the selling price of a home
- Unsupervised learning
 - Glean insights from unlabeled data
 - For example, use clustering to identify groups of customers with common traits

Supervised

col1	col2	col3	col4	label
34	1024	986	12	0
4	532	113	8	1
602	234	22	35	0
12	66	101	752	0
884	2192	73	618	1

Unsupervised

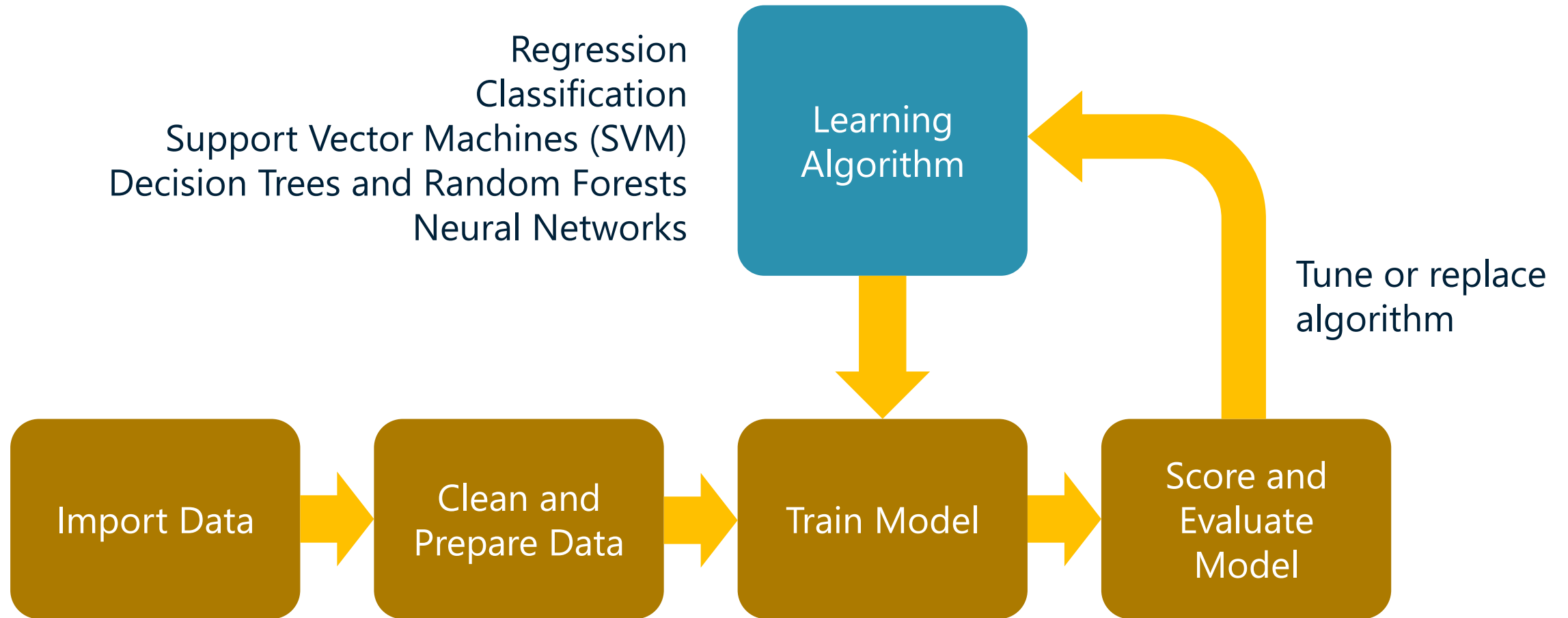


Scikit-Learn

- Machine-learning library for Python programmers
 - Free, open-source, and commercially usable under BSD license
- Supports classification, regression, clustering, dimensionality reduction, model selection, hyperparameter tuning, data preprocessing, and more
- Used in conjunction with other popular Python libraries such as NumPy, SciPy, Matplotlib, and Pandas
- <http://scikit-learn.org>



Building a Supervised-Learning Model



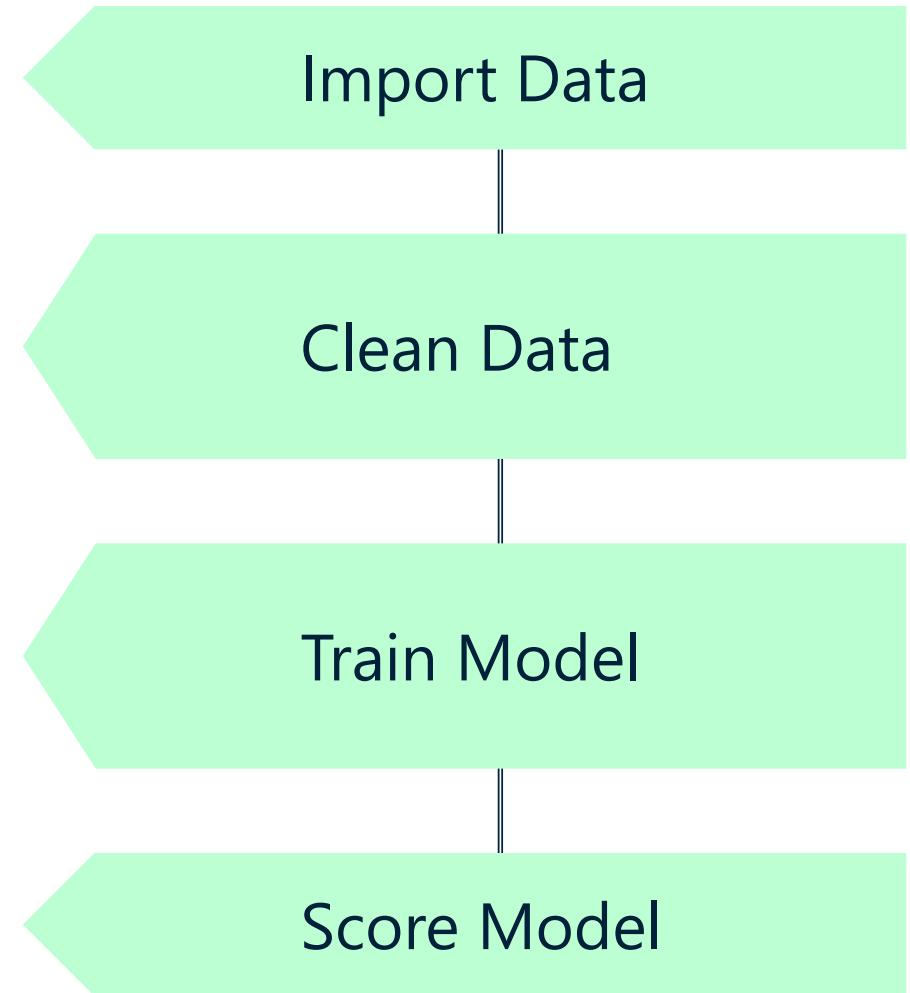
Supervised Learning with Scikit

```
# Load a dataset
df = pd.read_csv('arrivals.csv')

# Remove duplicate rows and rows with missing values
df = df.drop_duplicates()
df = df.dropna()

# Select a learning algorithm and train the model
model = LinearRegression()
model.fit(x_train, y_train)

# Score the model
accuracy = model.score(x_test, y_test)
```



Demo

Sentiment Analysis with Neural Networks



Comparing embedding vectors

- Determine similarity between two text samples
 - E.g., Compare document with query
 - Generate embeddings for each
 - Compute the similarity between embedding vectors
- Compute Dot product
 - Use cases: similarity-based systems
 - Semantic-search systems
 - Recommender
 - Deduplication

Comparing embeddings with dot product

```
from openai import OpenAI
import numpy as np

client = OpenAI()

x = client.embeddings.create (
    model='text-embedding-3-large', input='Jeff is a common name'
).data[0].embedding

y = client.embeddings.create (
    model='text-embedding-3-large', input='My name is Jeff'
).data[0].embedding

similarity = np.dot(np.array(x), np.array(y))
similarity

#0.6930581860828994
```

Cosine similarity

- Compute cosine of angle between two embedding vectors
 - Dot product of vectors / product of lengths
 - Depends only angle of vectors
 - *Not* on magnitude
- Advantages
 - Recommended by OpenAI
 - High cosine similarity for two texts with similar content but different lengths
- Results between -1 and 1
 - Proportional vectors = 1
 - Orthogonal vectors = 0
 - Opposite vectors = -1

Cosine Similarity

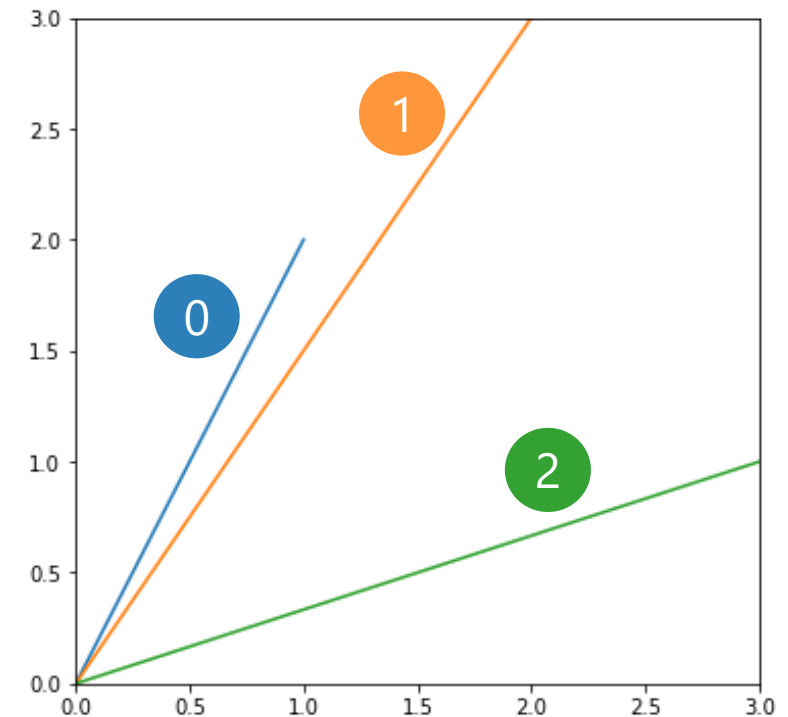
Question: Is row 1 more similar to row 0 or row 2?

Answer: Plot each row as a vector and compute the **cosines of the angles** between vectors

$$\cos \Theta_{01} = 0.992$$

$$\cos \Theta_{12} = 0.789$$

0	1	2
1	2	3
2	3	1



Computing Cosine Similarity

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
data = [[1, 2], [2, 3], [3, 1]] # Three rows of two columns each  
cosine_similarity(data) # Generates a 3x3 array of cosines
```

	0	1	2
0	1.000	0.992	0.707
1	0.992	1.000	0.789
2	0.707	0.789	1.000

OpenAI Python Cosine_similarity

- OpenAI Python 0.28.1 provided method to compute cosine similarity
 - Defined in module *openai.embeddings_utils*
 - <https://learn.microsoft.com/en-us/azure/ai-services/openai/tutorials/embeddings?tabs=python-new%2Ccommand-line#import-libraries>

```
import openai
import numpy as np
from openai.embeddings_utils import cosine_similarity
```

OpenAI Python Cosine_similarity

- OpenAI no longer includes *openai.embeddings_utils*
 - As of Python 1.x
- Official Open AI embedding models normalized to unit length of 1
 - Means cosine similarity is exactly equal to dot product
 - <https://developers.openai.com/api/docs/guides/embeddings>
- Can still define equivalent manually

```
def cosine_similarity(a, b):  
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```


Comparing embeddings with cosine similarity

```
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

x = client.embeddings.create(
    model='text-embedding-ada-002', input='Jeff is a common name'
).data[0].embedding

y = client.embeddings.create(
    model='text-embedding-ada-002', input='My name is Jeff'
).data[0].embedding

similarity = cosine_similarity(np.array(x), np.array(y))
similarity
#0.9002450768022142
```

Summary and Conclusions

- Large Language Model (LLM)
 - Capable of performing variety of NLP tasks
 - GPT-5 trained on trillions of parameters
- Tokenization
 - Foundational step in preparing text for input into language models
 - Tokens generated by splitting unstructured text into smaller units
- Embedding
 - Encodes tokens as vector of floating-point numbers
 - Representation corresponds to semantic meaning of text
- Transformers
 - Process embedding vectors to ultimate produce output
 - More on this in the next talk